Lecture 10
Part 1 (Linux) Review

Comp 206 – Introduction to Software Systems – Fall 2026

Mathieu Bérubé-Vallières

McGill University

October 1st, 2026

Note on study sessions

- If you do not see the TAs, please check in ENGTR 3104

Bérubé-Vallières © 2026
See last slide for complete credits

Note on MCQ:

- Due to Crowdmark limitations, MCQs with multiple correct answers are implemented as follows (equivalent):

Note on MCQ:

- Due to Crowdmark limitations, MCQs with multiple correct answers are implemented as follows (equivalent):

Winter 2026 exams (didn't use Crowdmark)

Q2 (2 points): Consider the octal permissions 316 on a file owned by user `alice` and `group` users. Suppose that user `bob` belongs to the group users, but that the user `claire` does not belong to the group users. Which of the following statements are true (Select all that apply)

a) `alice` can read the file
b) `alice` can write to the file
c) `bob` can execute the file
d) `claire` can read the file

Fall 2026 exams (Crowdmark)

Consider the octal permissions 316 on a file owned by user `alice` and `group` users.
Suppose that user `bob` belongs to the group users, but that the user `claire` does not belong to the group users. Which of the following statements are true? **(0.5 points each)**

Q2) `alice` can read the file [a) True b) False]
Q3) `alice` can write to the file [a) True b) False]
Q4) `bob` can execute the file [a) True b) False]
Q5) `claire` can read the file [a) True b) False]

Systems development, Linux and Bash

Bérubé-Vallières © 2026
See last slide for complete credits

Systems development

- A set of *interconnected*, *communicating* software artifacts, together with their specifications, configuration files, and documentation is called a **software system**.
- This course is an introduction to building large software
  - We discuss essential tools used by systems developers
    - Linux, Command-line, Bash, git, C…

Bérubé-Vallières © 2026
See last slide for complete credits

Linux (1/2)

- Operating system:
  - Manages the computer's resources
  - Insulates applications running on a computer from the hardware and each other
    - This includes providing programmers with system libraries to perform OS-level operations
  - Insulates the user from the complex inner workings of the computer
    - This includes providing a GUI or command-line, and utilities/software to interact with peripherals, files, networks, etc.

Bérubé-Vallières © 2026
See last slide for complete credits

Linux (2/2)

- Linux is a family of free and open-source operating systems (called Linux distributions) based on the Linux kernel
  - Linux systems are called Unix-like, because much of its design is based on principles established in the Unix family of operating systems
- In this module, you have learned to:
  - Use the command-line on Linux to read, write, and manage files
  - Compose atomic command-line programs into pipelines
  - Write bash scripts to automate tasks relating to software development
  - Use git to track modifications to bodies of source code
    - (You now know how to clone, add, commit and push: We will talk about more git features later in the course)

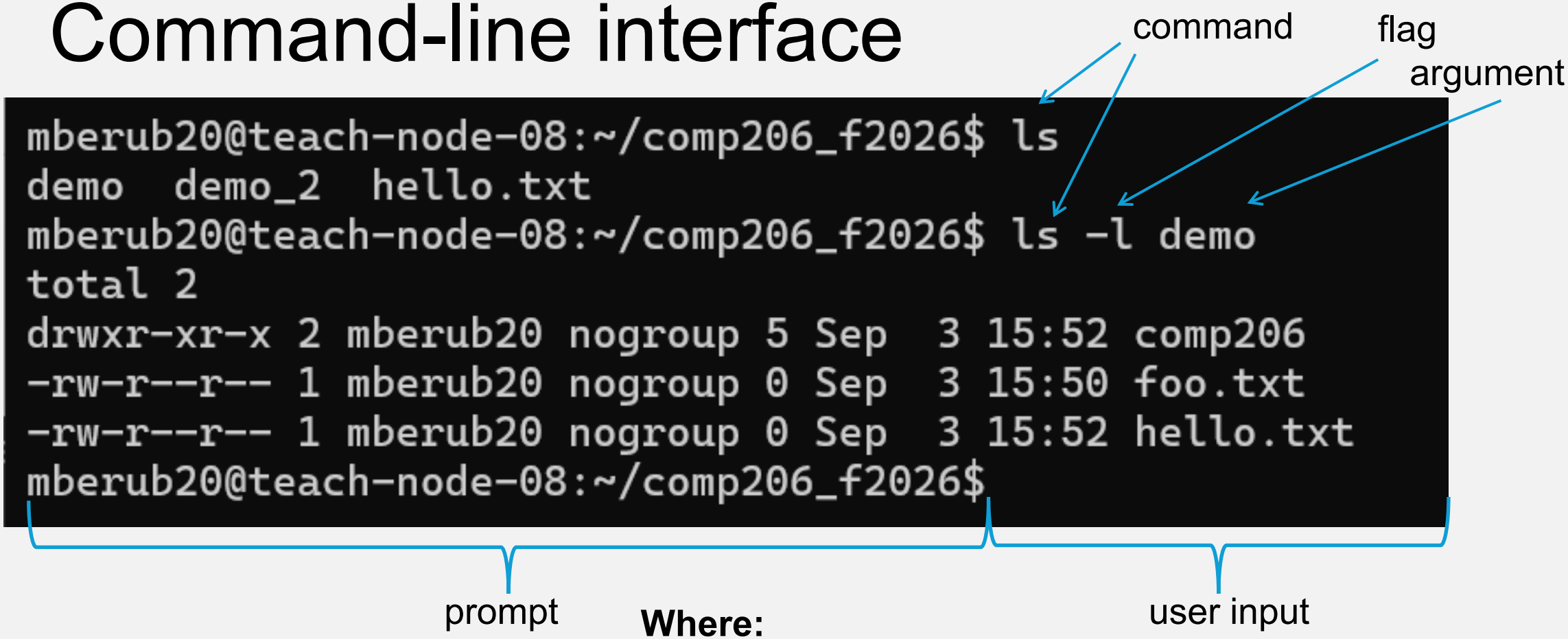
Bérubé-Vallières © 2026
See last slide for complete credits

Shell

- The *shell* is a program that provides users access to the system on which it runs
  - It provides a way to get user input (i.e. the *command-line*), displays OS information, and stores session information (e.g. the previous commands you ran)
- The shell we use in this class is *Bash*

Bérubé-Vallières © 2026
See last slide for complete credits

Command-line interface



Input format:
Command flags arguments

(order of flags and arguments is not always strict)

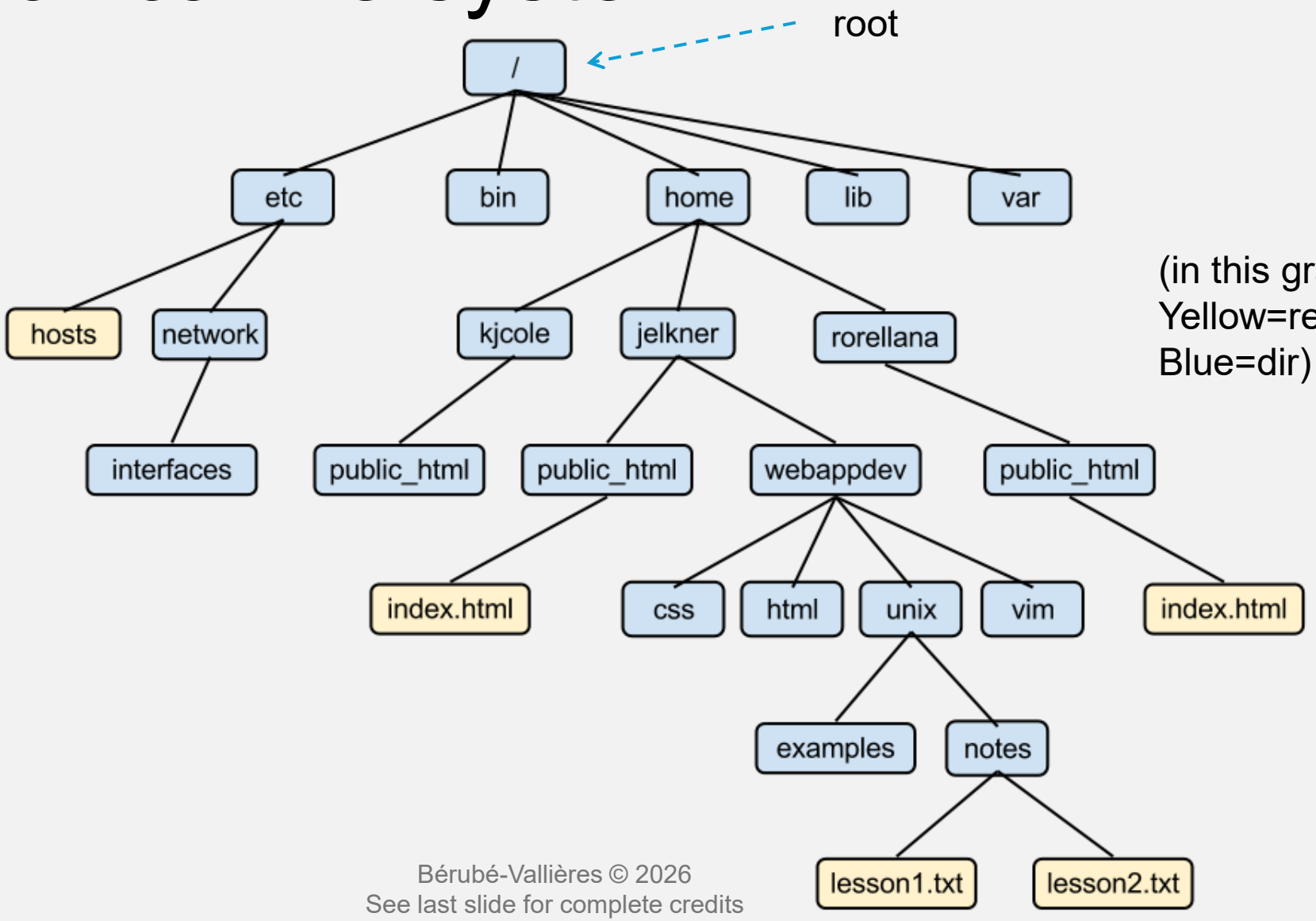
Where:

- Command: The program to run
- Flags (also called switches or options): modifies behaviour of the command
- Arguments (also called parameters): input passed to the command

Unix file system

Bérubé-Vallières © 2026
See last slide for complete credits

Hierarchical file system:



(in this graph:
Yellow=regular file
Blue=dir)

Bérubé-Vallières © 2026
See last slide for complete credits

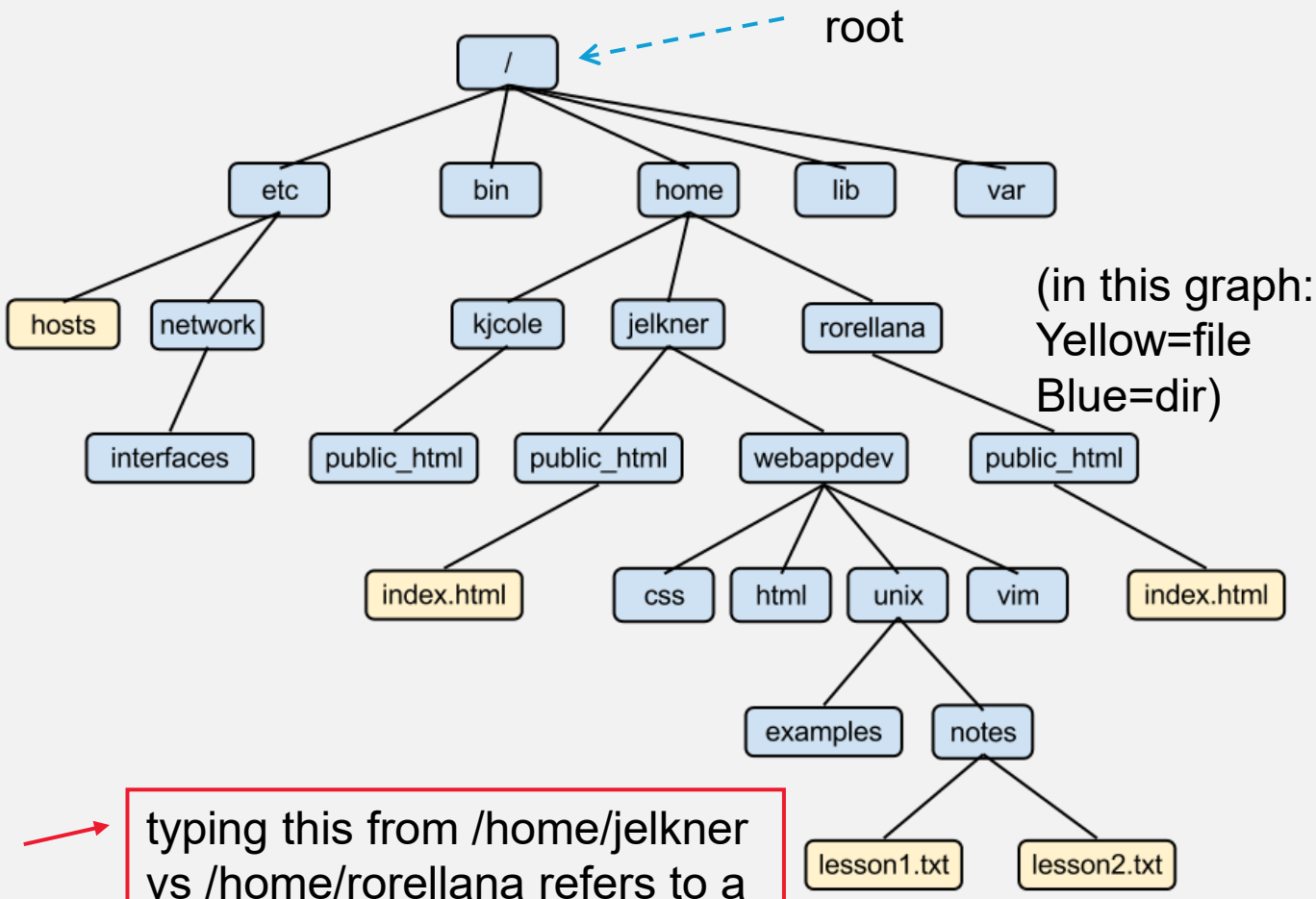
The truth about Unix files and directories

- Philosophy: Everything is a file
- Directories are simply a special type of file that contains a list of index nodes (*inodes*)
  - Index nodes contain metadata about a file, including whether it's a regular file or directory (or other special types), who owns it, the permissions, etc., and the location of the file on the disk
  - Will not discuss inodes much, but good to keep in mind
    - We will sometimes use "regular file" when we want to talk specifically about "standard" files
- Unix does not impose any file structure (file format) to regular files. Their structure and the way to interpret them is entirely dependent on the software using them.
  - File extensions mean nothing and are *only* useful for the user. Different from Windows!

Bérubé-Vallières © 2026
See last slide for complete credits

Location in file system

- The root is the top folder of the OS
  - Symbol: `/`
- Two ways to refer to a location in the file system:
  - **Absolute path**: A string that begins at the root
    - `/etc/hosts`
    - `/home/jelkner`
  - **Relative path**: A string that begins at the current location
    - `public_html/index.html`
    - `lesson1.txt`

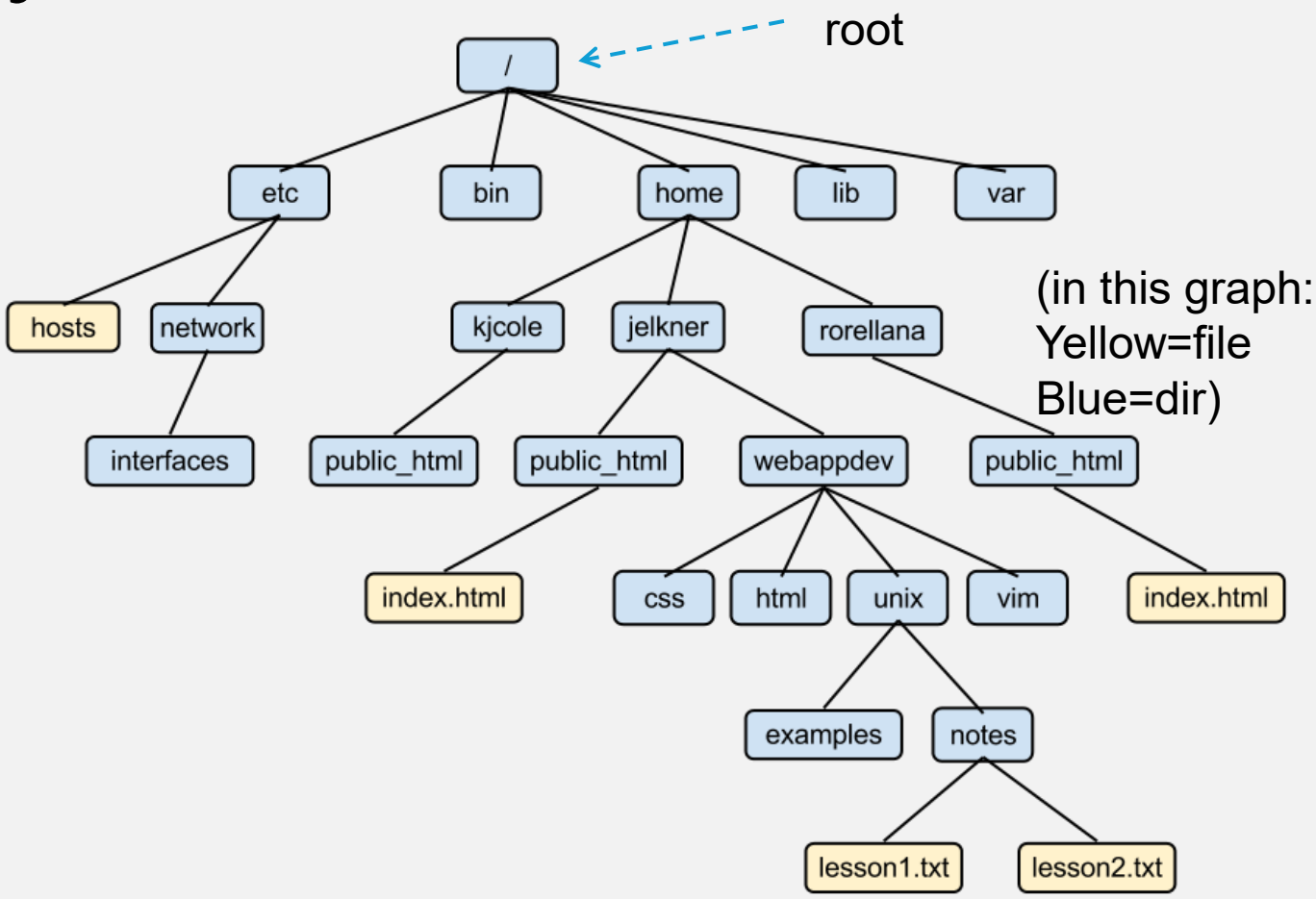


typing this from /home/jelkner vs /home/rorellana refers to a different location!

Bérubé-Vallières © 2026
See last slide for complete credits

Other file system symbols

- The home is the top folder in the user's directory tree
  - Special symbol: `~`
    - This is a wildcard that gets expanded to the value of environment variable `$HOME`
    - e.g: If `$HOME` contains `/home/kjcole`, then `~/public_html` expanded to `/home/kjcole`
- Each directory contains a hard link to itself and its parent:
  - The directory itself: `.`
  - The parent directory: `..`
    - e.g: `../unix/examples`



Bérubé-Vallières © 2026
See last slide for complete credits

Commands

Bérubé-Vallières © 2026
See last slide for complete credits

Commands

- Command syntax
  - command -[options] argument1 argument2 …
- Command-line prompt:
  - username@hostname:currentdirectory$ [enter command]
- man: **man**ual pages for linux commands
  - Example: `man ls`
  - An exception is built-in shell commands, use `man builtins` to see the list

Bérubé-Vallières © 2026
See last slide for complete credits

Commands

- pwd: **p**rint **w**orking **d**irectory
- cd: **c**hange **d**irectory
- ls: **ls**t directories
  - -l flag: long format
  - -a flag: show hidden files
  - Can combile flags: e.g. –la
  - Can use multiple arguments (like most commands!):
    - ls dir1 dir2 file1…
- mkdir: **m**a**k**e **dir**ectories (if does not exist)

Bérubé-Vallières © 2026
See last slide for complete credits

Commands

- cp: **c**o**p**y files
  - -r: copy recursively (needed to copy directory!)
  - Can copy a source file/dir to a destination, or copy source(s) to a directory
- mv: **m**o**v**e files
  - Can move a source file/dir to a destination, or move source(s) to a directory
- rm: **r**emo**v**e files
  - -r: copy recursively (needed to copy directory!)
- cp and mv overwrite without asking! rm removes files for good (no recycle bin!)

Bérubé-Vallières © 2026
See last slide for complete credits

Text-related commands (1/4)

- touch: make an empty file (if does not exist)
- cat: con**cat**enate files and display the concatenated result
- more: page through a file
  - let's you show "more" text on the screen
  - `less`: updated version of `more`
    - `more` doesn't let you go backwards, `less` does
- diff: show the **diff**erence between two files

Bérubé-Vallières © 2026
See last slide for complete credits

Text-related commands (2/4)

- `cut -d <delimiter> [file]`
  - `cut` command is used to extract specific sections of text from each line of a file or standard input. It works well with structured text (like columns or delimited data). You can select output by `-f` to extract fields separated by a delimiter
  - Example: `cut -d ',' -f 1,3 data.csv`
- `wc [-clw] [file]`
  - `wc` (word count) simply counts the number of words, lines, and characters in the file(s).
  - The three options, `clw`, stand for character, line, and word respectively, and tell `wc` which of the three to count.
- Example: `wc -l data.csv`

Bérubé-Vallières © 2026
See last slide for complete credits

`cut` example with CSV file

- CSV = Comma Separated Values
- File is composed of units of data that are separated from each other by commas or newlines

```
data1,data2,data3,data4
data5,data6,data5,data8
```

- Each row is called a “record”
- Each unit of data in a row is called a “field”



- The above example stores information about people: first name, last name, age

Bérubé-Vallières © 2026
See last slide for complete credits

`cut` example with CSV file

- Say that `users.csv` stores:

```
fname,lname,age
Mary,Brown,18
John,Smith,98
Mary,Johnson,24
```

- We can extract the lname column with `cut`

```
cut -d ',' -f2 users.csv
```

- Output:

```
lname
Brown
Smith
Johnson
```

delimiter is a comma

second field

Bérubé-Vallières © 2026
See last slide for complete credits

Text-related commands (3/4)

- `sort [options] [file]`
  - `sort` command is used to sort data in file or standard input (in case of the absence of the file argument)
  - `-n` option treats sorting key as numeric (Important in ensuring 5 comes before 25, etc.)
  - `-r` sorts in descending order
  - `-o outputfile` writes the sort output to output file. Otherwise, sort will write to the standard output.
- `uniq [file]`
  - `uniq` (unique) command filters out repeated lines from a file or input stream. It removes consecutive duplicate lines and prints only one instance of each.
  - `-c` option prefix each line with the number of occurrences
  - **Important:** uniq only works properly on sorted input, because it only compares adjacent lines

Bérubé-Vallières © 2026
See last slide for complete credits

Text-related commands (4/4)

- `grep` is used to search for the patterns in files.
- `grep [options] STRING FILE_LIST`
- Some common options include:
  - `-i` : ignore case
  - `-n` : display the line number along with the line on which a match was found
  - `-c` : report only a count of the number of lines containing matches
  - `-v` : invert the search, displaying only lines that do not match
  - `-l` : list filenames, but not lines, in which matches were foun

Bérubé-Vallières © 2026
See last slide for complete credits

grep: example

- Consider the following content of file `students.txt`:

Alex
Marc
Michael
Ting
Juan
Jeremy
Jessica
Yannick
Nicolas
Jean-Sebastien
Nadeem

```
$ grep 'Je' students.txt
Jeremy
Jessica
Jean-Sebastien
```


Bérubé-Vallières © 2026
See last slide for complete credits

Tar

- An archive is a collection of files combined into one file.
    - Being one file, archives are easier to manipulate (move, store, copy, backup, etc).
    - Archives are often compressed, so they require less space.
- The two most common archive tools used on Unix systems is tar and gzip (gunzip).
- Tar allows the manipulation (creation, extraction) of archive files.
    - Convention:
        - A file ending with the .tar extension is a tar archive file.
        - A file ending with the .tgz extension is a compressed (gzipped) tar archive file.
- Switches:
    - -c : create a new tar archive
    - -r : update the tar archive
    - -x : extract from the tar archive
    - -f: specifies the archive file name.
    - -v: activates verbose mode, which means the tar command will output lots of information.
    - -z: allows you to compress the archive (the archive is compressed/decompressed using gzip).

Bérubé-Vallières © 2026
See last slide for complete credits

ln: hard and symbolic links

- `ln` and `ln -s`
- The ln command can be used to create links to files and folders.
  - Hard link: `ln /path/file link_name`
  - Soft link: `ln -s /path/file link_name`
- When creating a hard link, you are simply giving another name to a file (it shows up separately on an `ls -a` direct pointer in directory)
  - The link will point to the same physical space on the disk.
  - A file can only be deleted once all its hard link are deleted.
- When creating a symbolic link (using `ln -s`), a new file is created (an indirect pointer in directory)
  - The new file automatically redirects to the target file.
  - Symbolic links can be created across volumes (or disks).
  - Deleting a symbolic link does not affect the target file.

Bérubé-Vallières © 2026
See last slide for complete credits

Wildcards

Bérubé-Vallières © 2026
See last slide for complete credits

Wildcards

- A wildcard is a symbol used to replace one or more characters in a filename. In Bash (and POSIX), these are:
- The “asterisk”: Any pattern
  - `*.doc`
- The “question mark”: Any single character
  - `?at.doc`
- The “square brackets”: Any character within the brackets
  - `cat.d[aoz]c`

Bérubé-Vallières © 2026
See last slide for complete credits

Wildcards

- A wildcard is a symbol used to replace one or more characters in a filename. In Bash (and POSIX), these are:
- The “asterisk”: Any pattern
  - `*.doc`
- The “question mark”: Any single character
  - `?at.doc`
- The “square brackets”: Any character within the brackets
  - `cat.d[aoz]c`

Inside brackets…
- Two characters separated by hyphen are a range, for example [a-d] is equivalent to [abcd]
- You can put ! at the front to match anything not in the square brackets, for example, [!a] matches any single character except the letter a

Wildcards

`echo` is useful to see how a command gets expanded!

- A wildcard is a symbol used to replace one or more characters in a filename. In Bash (and POSIX), these are:
- The “asterisk”: Any pattern
  - `*.doc`
- The “question mark”: Any single character
  - `?at.doc`
- The “square brackets”: Any character within the brackets
  - `cat.d[aoz]c`

Inside brackets…
- Two characters separated by hyphen are a range, for example [a-d] is equivalent to [abcd]
- You can put ! at the front to match anything not in the square brackets, for example, [!a] matches any single character except the letter a

Filename expansion (globbing)

- The shell recognizes and expands wildcards patterns into the list of pathnames matching the pattern. This is called *globbing*
- As the shell always does this when interpreting your input, this works for all the commands you've seen
- For example, if you type `ls *.doc` in a directory with the files on the right, you will see all the files that end with .doc files in your directory. If you type `rm ?at.doc` you will remove cat.doc and bat.doc

john.doc
cat.doc
bat.doc
cat.dac
file.dat
cat.dpt
file.txt

Bérubé-Vallières © 2026
See last slide for complete credits

Who globs

- The shell expands the wildcard patterns before giving them as arguments. For example, if you type `ls *.doc` the shell expands it into `ls john.doc cat.doc bat.doc`
  - The command has no idea that you used wildcards, the shell is the one doing the work!
- You can see this by yourself with the echo command: Try echo `ls *.doc` in a directory with doc files.

john.doc
cat.doc
bat.doc
cat.dac
file.dat
cat.dpt
file.txt

Bérubé-Vallières © 2026
See last slide for complete credits

If wildcards can’t be matched

- If the wildcards can’t be matched, the shell itself won’t do anything with them or send an error message, and the wildcards will simply be sent to the command literally
- For example, in a directory with no `.doc` files, echo `ls *.doc` will echo back `ls *.doc` and `ls` will think that you’re trying to open a file literally called `*.doc`
- This can cause unexpected results

Bérubé-Vallières © 2026
See last slide for complete credits

Command substitution

- The shell can also substitute the outputs of commands
- Syntax: `$(command)`
- Example: `touch $(date +%Y-%m-%d).txt`
  - The shell first runs `date +%Y-%m-%d`
    - `date` is a command that outputs today's date. `+%Y-%m-%d` specifies that the format should be YYYY-MM-DD
  - The output of this command today is `2026-09-10`
  - The shell replaces `$(date +%Y-%m-%d)` by `2026-09-10`
    - i.e. the command becomes `touch 2026-09-10.txt`
  - `touch` creates file `2026-09-10.txt`

Bérubé-Vallières © 2026
See last slide for complete credits

Arithmetic expansion

- It is possible to evaluate arithmetic expressions in Bash by using $((expr))
- Examples:
  - `echo $(( 5 + 3 ))`
  - `result=$(( $1 + $2 ))`
    - Adds the contents of $1 and the contents of $2
  - `echo $(( var++ ))`
    - Outputs `$var` and increases its value by 1 (post-increment)

Bérubé-Vallières © 2026
See last slide for complete credits

Quotes

- Single quotes (`'`) are used to preserve the literal value of characters within the quotes
- Double quotes (") are used to preserve the literal values of characters within the quotes, except $, ` and \
  - i.e. variables, command substitution and escape characters are still processed
- In both cases, the space character is preserved. For example, compare
  - `mkdir John Smith`
    - creates two directories, “John” and “Smith”
  - `mkdir "John Smith"`
    - creates one directory, “John Smith”

Bérubé-Vallières © 2026
See last slide for complete credits

Escape characters

- The backslash (\) is Bash's escape character
- It preserves the literal value of the next character that follows
  - For example: \" is the literal single quote character, as opposed to the start of a quote
- With some commands and some characters, it can instead toggle on a special meaning for the character:
- For example, when using `echo -e`
  - `\n` means newline
  - `\r` means return
  - `\t` means tab

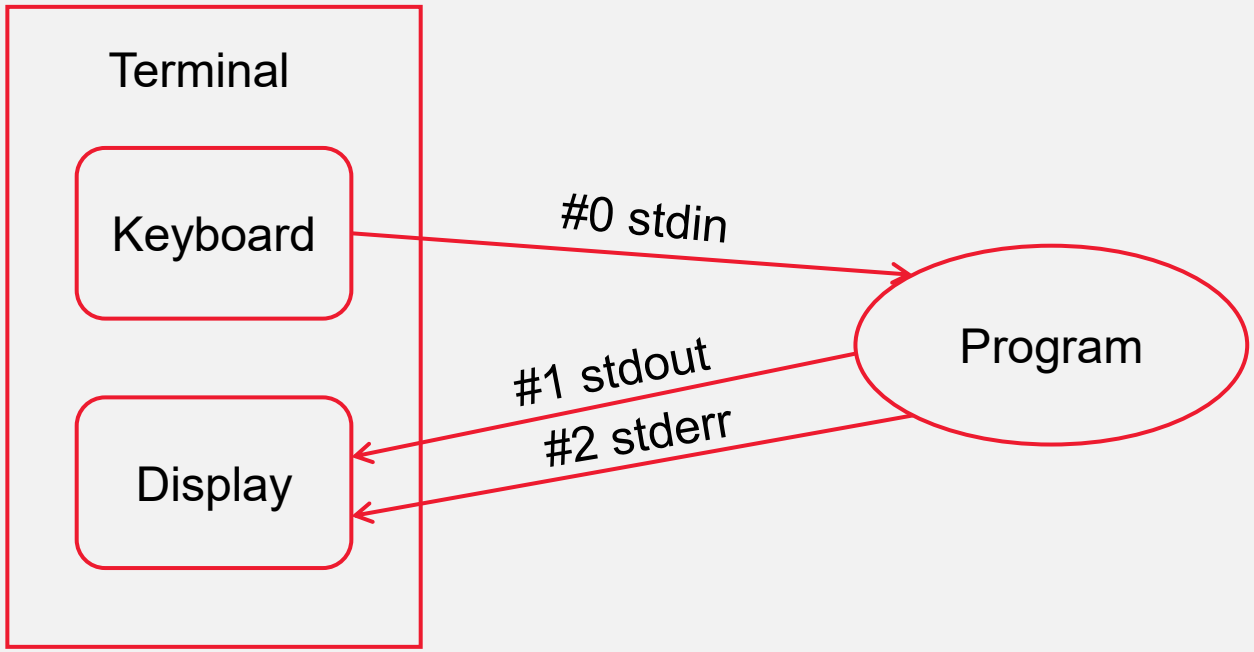
Bérubé-Vallières © 2026
See last slide for complete credits

Redirection

Bérubé-Vallières © 2026
See last slide for complete credits

Programs and standard stream

- Streams:
  - `stdin`: where program reads its input data
  - `stdout`: where program writes its output data
  - `stderr`: where program writes its error messages
- By default:
  - stdin goes from keyboard to program
  - stdout and stderr goes from program to display
- But can redirect these streams!



Bérubé-Vallières © 2026
See last slide for complete credits

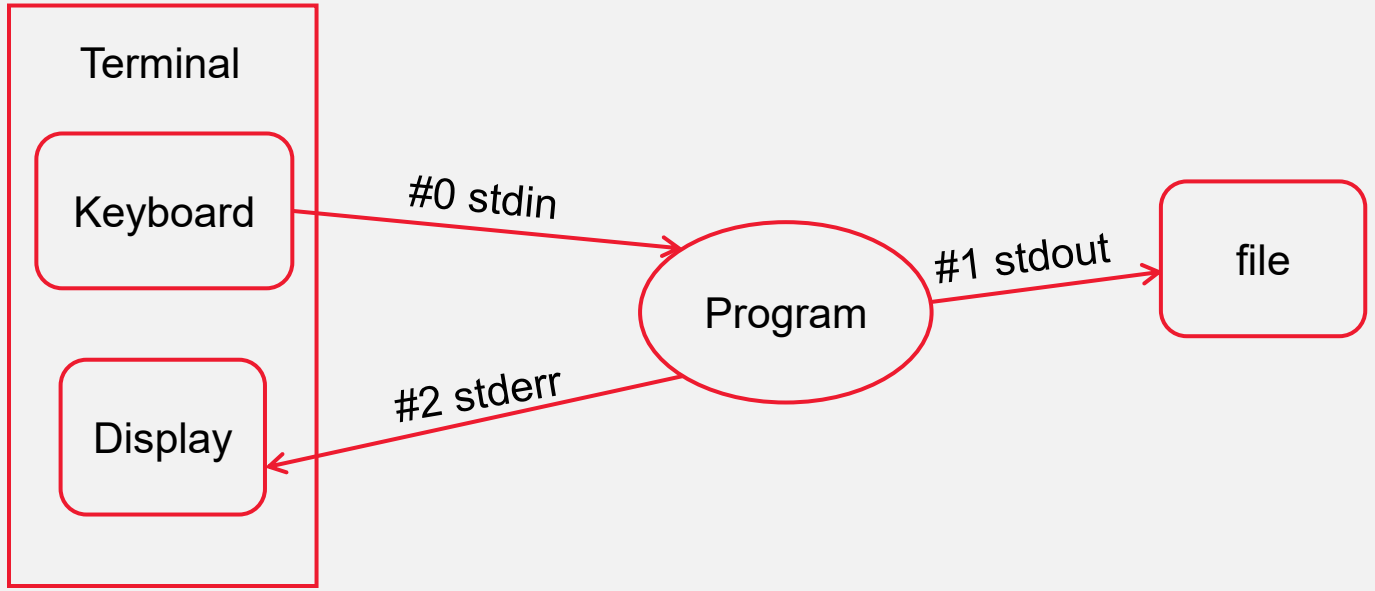
Redirecting stdout to a file

Redirect stdout to a file instead of display:

- Symbol: `>`
- Ex: `ls -la > list.txt`

Variant: append to existing file instead of overwriting

- Symbol: `>>`
- Ex: `ls -la >> list.txt`

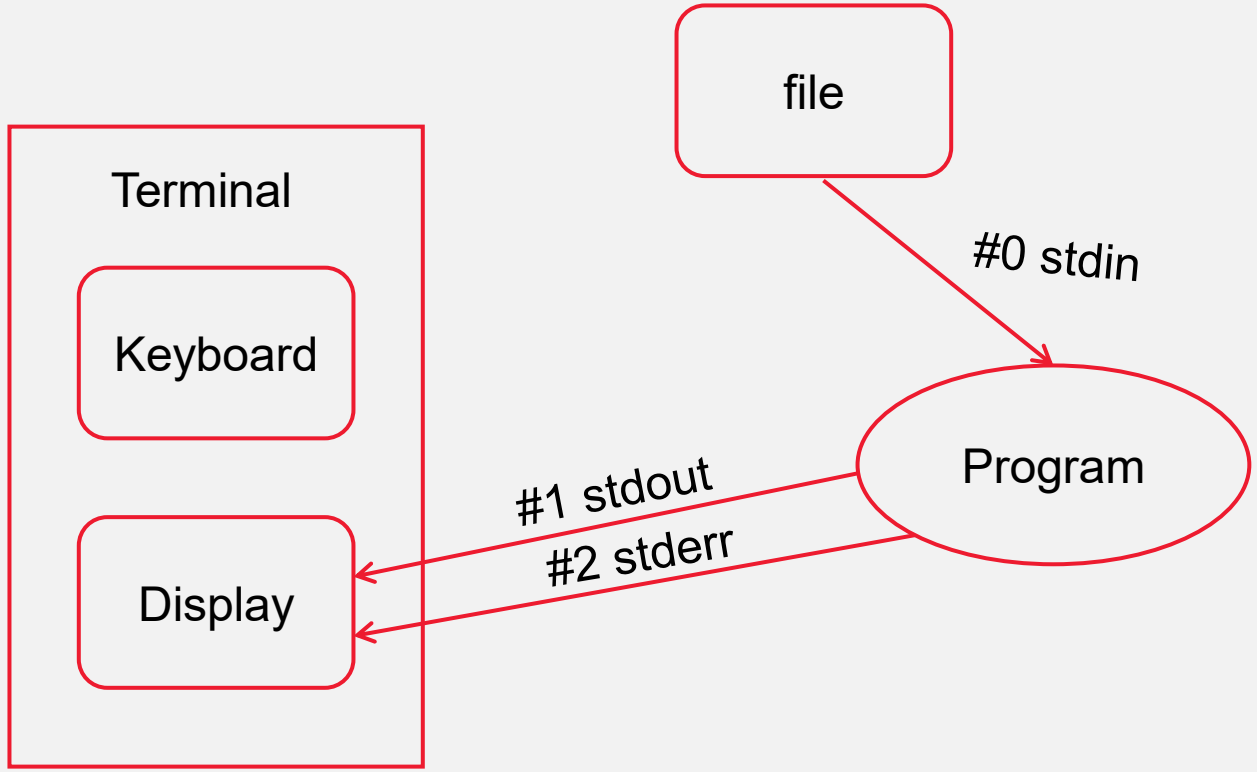


Bérubé-Vallières © 2026
See last slide for complete credits

Redirecting stdin from a file

Redirect stdin from a file instead of keyboard:

- Symbol: `<`
- Ex: `my_program < input.txt`

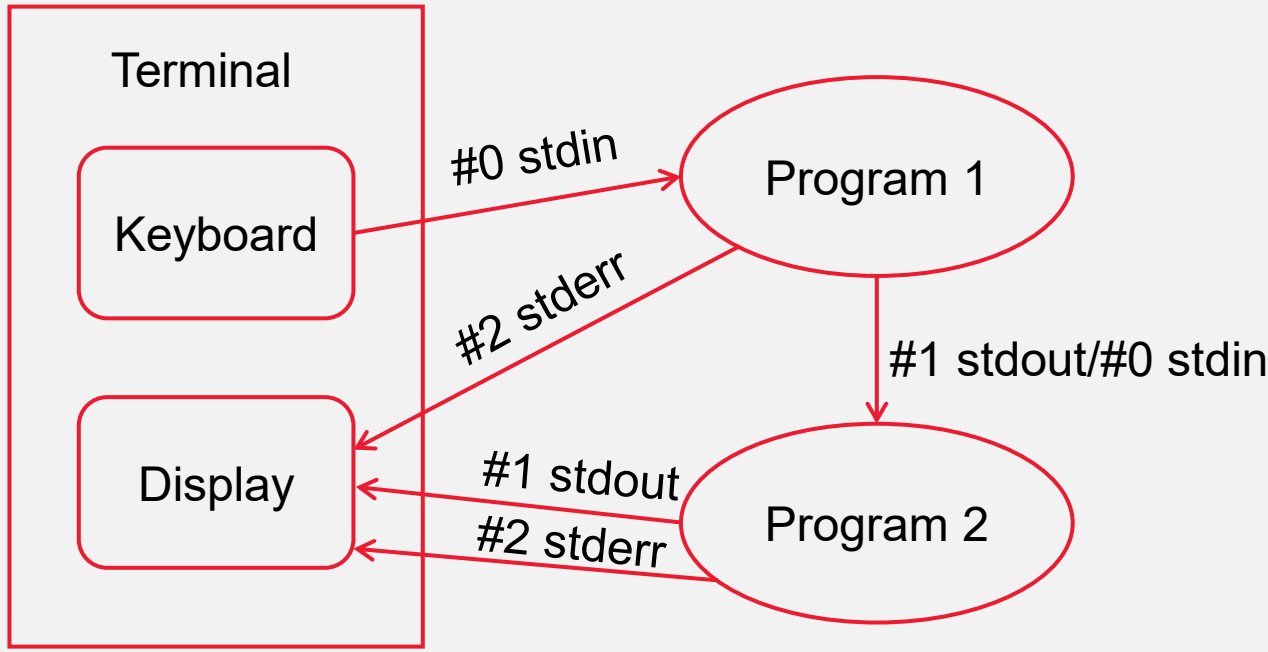


Bérubé-Vallières © 2026
See last slide for complete credits

Connecting stdin/stdout of two programs: Piping

Redirecting stdout from a program to stdin of a program has a special name: *Piping*

- Symbol: `|` (called *pipe*)
- Ex: `ls -l | more`



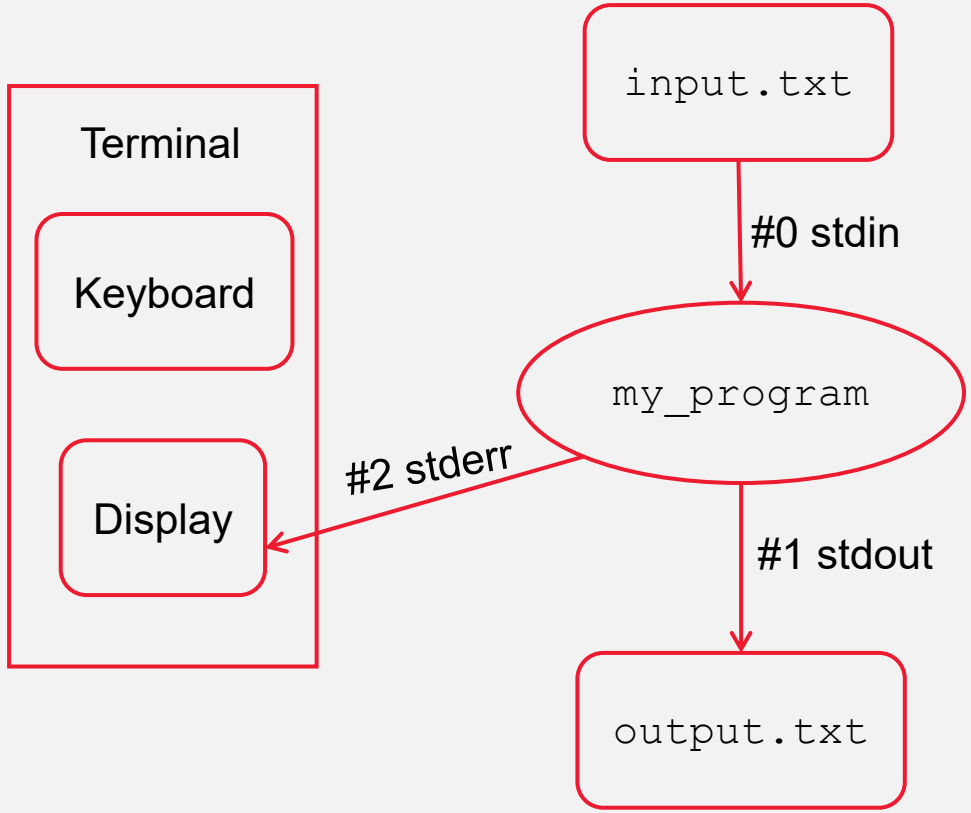
Bérubé-Vallières © 2026
See last slide for complete credits

Chaining!

Redirection and piping can be chained together to make complex commands!

- Ex: Get input from file, and output to another file

```
my_program < input.txt > output.txt
```



Bérubé-Vallières © 2026
See last slide for complete credits

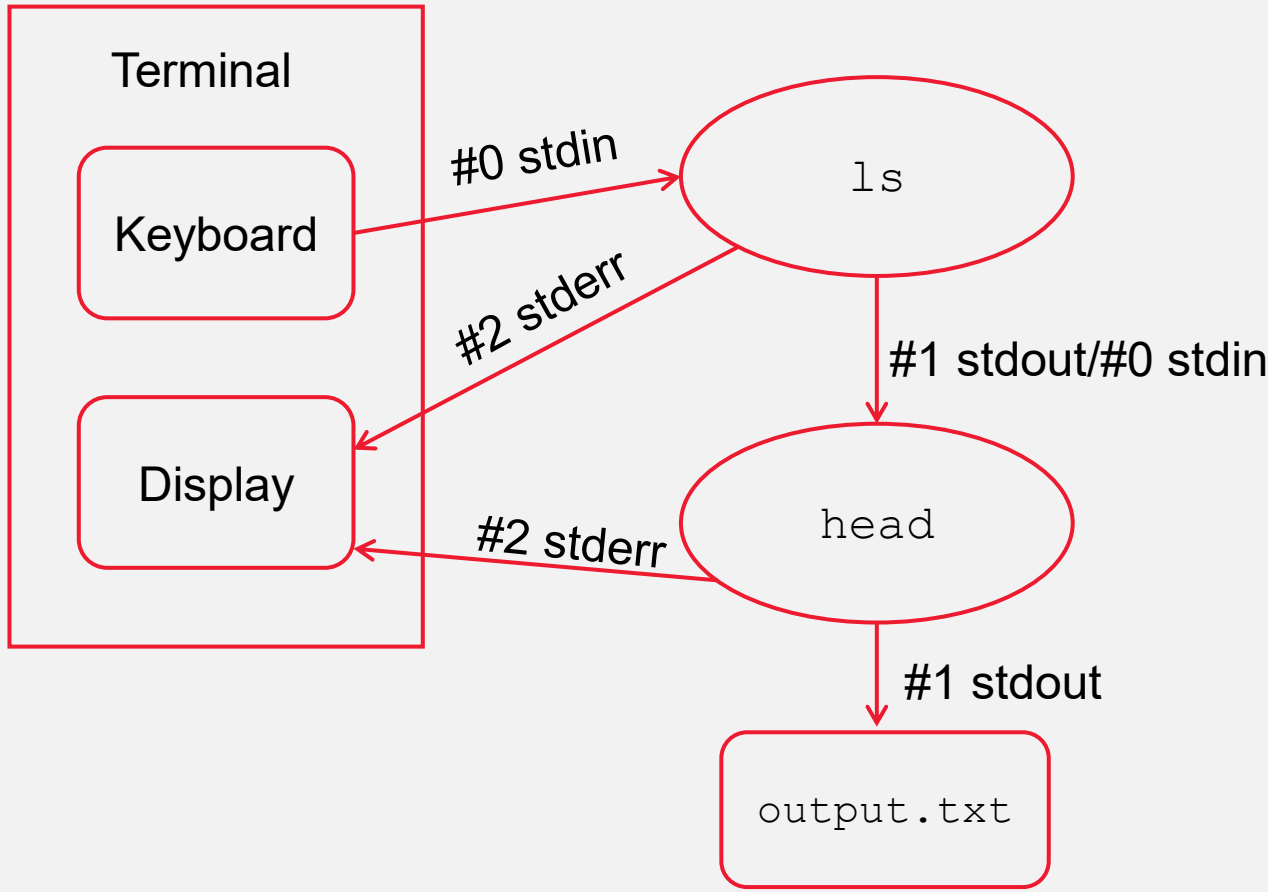
Chaining!

Redirection and piping can be chained together to make complex commands!

- Ex: Pipe and redirect output to file

```
ls -l | head > output.txt
```

(`head` shows the first 10 lines)



Bérubé-Vallières © 2026
See last slide for complete credits

Another chaining example

- Text stream processing commands are often chained together with pipes (|) to build powerful text-processing workflows.
- Example:

```
cat data.txt | cut -f2 | sort | uniq -c | sort -nr
```


Bérubé-Vallières © 2026
See last slide for complete credits

One more chaining example

- `cat $(ls *.log | tail -n5) >> text.out`
  - Assume we have files `f1.log f2.log … f20.log`
  - Shell interprets `ls *.log | tail -n5` first
    - Expands `ls *.log` to `f1.log f2.log … f20.log`
    - `ls` gives back the list of files
    - This is piped to `tail`
      - `tail` shows the last 10 lines by default, but the `-n5` flag here makes it show only the last 5 lines
  - Shell substitutes `$(ls *.log | tail -n5)` by the output of the command, i.e. `f16.log f17.log … f20.log`
  - cat concatenates the concents of the files (e.g. `f16.log f16.log … f19.log)` and the output is redirected to text.out

Bérubé-Vallières © 2026
See last slide for complete credits

Vim

Bérubé-Vallières © 2026
See last slide for complete credits

Vim

- Powerful text editor designed to efficiently write text from a terminal
- Preinstalled on most Linux distributions
- Modal editor
  - Keys have different functions depending on current mode
  - (compare to editors like Notepad where typing always inserts text)

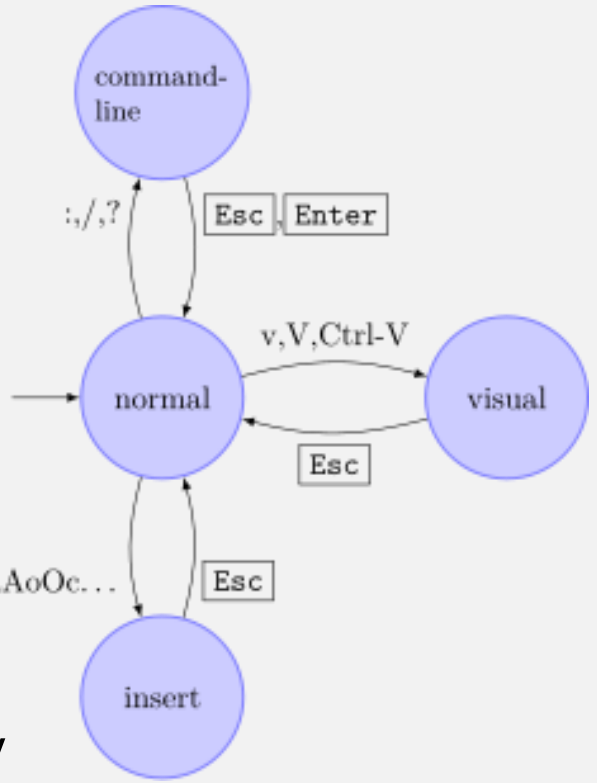
Bérubé-Vallières © 2026
See last slide for complete credits

Vim modes

- Instead of a menu system, Vim uses *modes*

The most important modes are:

- Insert mode:
  - To edit your text
  - Can press any keyboard characters
- Normal mode (also known as command mode):
  - Enter editor commands
    - Move around in the file, change modes, copy, undo, etc.
- Command-line mode:
  - Save, load, quit, search, etc.
- Visual mode:
  - Like normal modes, but movement keys highlight lines to apply editor commands on all of them



Bérubé-Vallières © 2026
See last slide for complete credits

Important commands (1/2)

To get help for a command
:help command

- Insert mode:
  - To get in insert mode, any of the following: i, I, a, A, o, O
    - i – insert text before the cursor | I – insert at start of line before first non-blank
    - a – append text after the cursor | A – append text at the end of the line
    - o – begin a new line below the cursor and insert text | O – same but above cursor
- Normal mode:
  - To get to to normal mode: Esc key
  - To move:
    - h, j, k, l to move left, down, up, right
      - (arrows also work but not recommended)
    - gg – go to first line | G – go to last line
      - (many others like w to go to next word, [(to go to next unmatched (, etc.)
  - To delete: dd, D, x, r
    - dd – delete a line | D – delete the rest of the line
    - x – delete a character | r – replace a character
  - To copy (yank) and paste a line:
    - yy or Y (equivalent) – yank | p - paste after cursor | P – paste before cursor
  - To undo and redo:
    - u (undo) and ctrl-r (redo)

Bérubé-Vallières © 2026
See last slide for complete credits

Important commands (2/2)

To get help for a command
:help command

- Command-line mode:
  - w, q, wq, q!, line number, e filename
    - :w – save file | :q – quit current window | :wq save and quit | :q! quit and discard changes
    - :number – go to a specific line number (For example, :14)
    - :e filename – Edit a file
  - To search: / (forward), ? (backwards)
    - Once you press enter, press n and N to get next/previous result

Bérubé-Vallières © 2026
See last slide for complete credits

Permissions

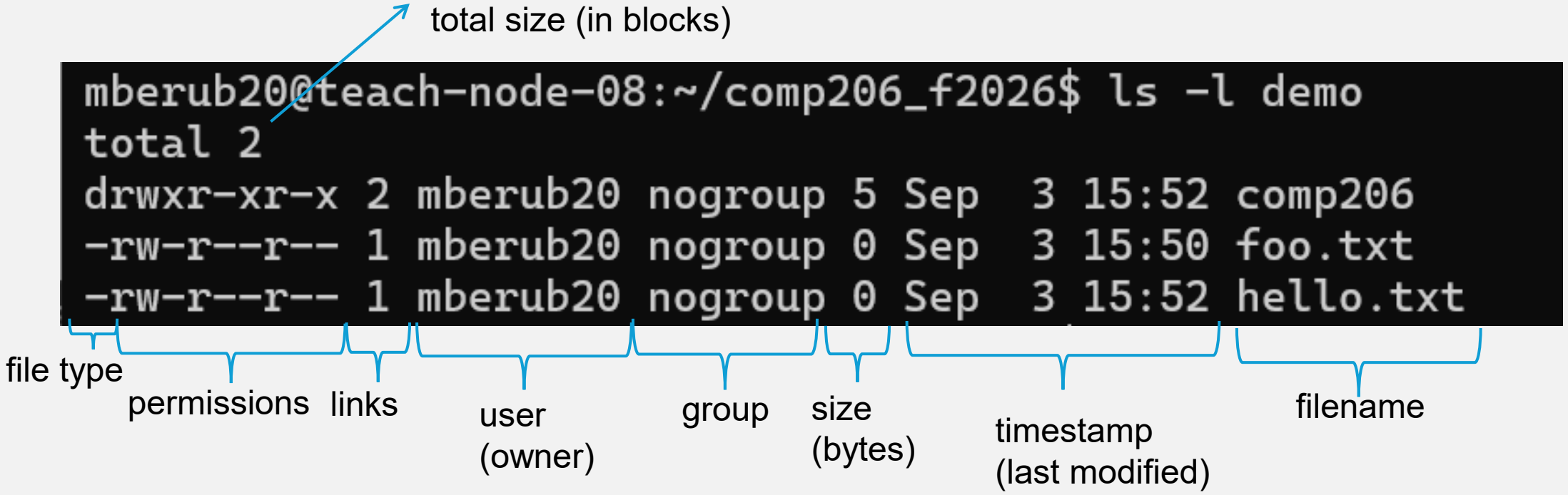
Bérubé-Vallières © 2026
See last slide for complete credits

Permissions

- All files are **owned** by a user and a group.
  - Usually, this owner is the user that created the file.
- Permissions on files exists at three level (classes):
  - **user**, **group** and **others**.
- Three types of rights can be given:
  - **read**, **write** and **execute**.
- Any combination of these rights must be given to these three levels.

Bérubé-Vallières © 2026
See last slide for complete credits

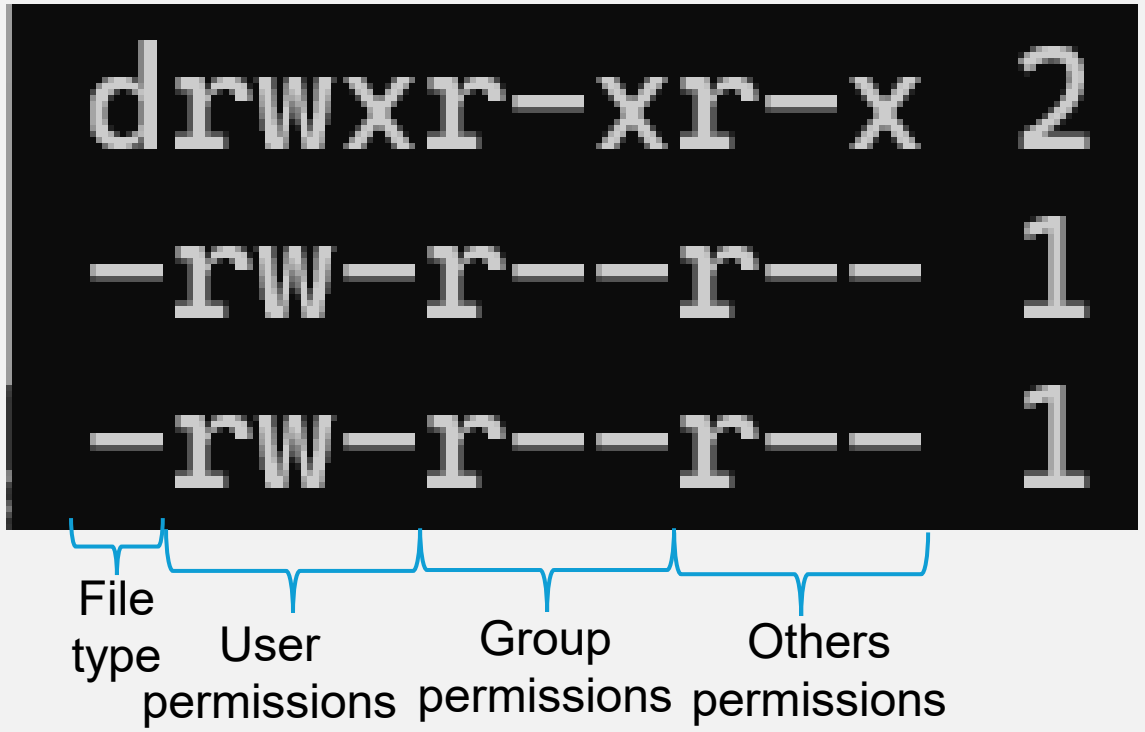
Recall: `ls -l`



Bérubé-Vallières © 2026
See last slide for complete credits

Symbolic notation

- The symbolic notation for permissions is displayed as a string of 9 characters (10 if including file type)
  - (0th : indicates if the file is a directory.)
  - 1st : indicates if the user that owns the file has read access to the file.
  - 2nd : indicates if the user that owns the file has write access to the file.
  - 3rd : indicates if the user that owns the file has execute access to the file.
  - 4th , 5th , 6th : indicates if the group owner has read, write or execute.
  - 7th , 8th, 9th : indicates if all other users have read, write or execute.



Order of precedence for permissions

- The order of precedence for permissions is
  - User
  - Group
  - Others
- If a user is in multiple classes, the highest one applies
- Can create surprising situations:
  Given permissions “------rwx” of a file I own, can I read the file?
  - I cannot read the file
  - People in the group cannot read the file
  - Others will be able to read the file

Bérubé-Vallières © 2026
See last slide for complete credits

`chmod` command

- **Ch**ange **mod**e: Change file permissions
  - Can be used by owner or superuser (root)
- Who:
  - u : The user who owns the file (this means “you.”)
  - g : The group the file belongs to.
  - o : The other users
  - a : all of the above (an abbreviation for ugo)
- Permission
  - r : Permission to read the file.
  - w : Permission to write (or delete) the file.
  - x : Permission to execute the file (for directories, this is needed to enter them)
- Changes to
  - = : become
  - + : add
  - - : remove

Bérubé-Vallières © 2026
See last slide for complete credits

`chmod` command examples

- The syntax of the command is as follows:
  - `chmod who=permission files`

Here are a few examples of the chmod command:

- Give read permission to group
  - `chmod g+r file.txt`
- Give write/execute permission to you (user)
  - `chmod u+wx file2.txt`
- Remove all permissions from others
  - `chmod o= file3.txt`
- Give read/write permission only to user and group
  - `chmod ug=rw file4.* file2.txt`
- Give read only to user and execute only to group
  - `chmod u=r,g=x file4.* file2.txt` (no space after the comma)

Bérubé-Vallières © 2026
See last slide for complete credits

Octal representation of permissions

- Given a 9 character permission string:

rwx --- rw-

- We can equivalent represent it in binary

111 000 110

- Given the binary representation of permissions, we can convert it to *octal* (base 8) by converting each group of 3 bits to its corresponding digit as shown on the right

Symbolic	Binary	Octal
`---`	000	0
`--x`	001	1
`-w-`	010	2
`-wx`	011	3
`r--`	100	4
`r-x`	101	5
`rw-`	110	6
`rwx`	111	7

Bérubé-Vallières © 2026
See last slide for complete credits

`chmod` and octal representation

- `chmod` supports octal representation
  - Often used when want to set up all permissions at the same time
- Ex:
  - chmod 400 filename
    - Give user permission to read and no permissions to group and others
  - chmod 644 filename
    - Give user permission to read and write and only give permission to read to group and others
  - chmod 700 filename
    - Give user all permissions and no permissions to group and others

Symbolic	Binary	Octal
`---`	000	0
`--x`	001	1
`-w-`	010	2
`-wx`	011	3
`r--`	100	4
`r-x`	101	5
`rw-`	110	6
`rwx`	111	7

Bérubé-Vallières © 2026
See last slide for complete credits

Bash scripting

Bérubé-Vallières © 2026
See last slide for complete credits

Bash scripting

- Bash:
  - Interactive mode: Commands are read from command-line
  - Non-interactive mode: Commands are read from files
- Executing commands from files lets user or system automate common sequences of commands
- These files are called *scripts*
- Scripts are run in a similar way to any programming language
  - Scripting languages are *interpreted* (as opposed to *compiled*): code is executed directly without being compiled to machine code

Bérubé-Vallières © 2026
See last slide for complete credits

Shebang

- We can tell the OS which interpreter the file should be run with by using a *shebang*
  - Remember: filename extension is only a visual indicator for humans in Unix systems!
- The shebang is the very first line of the file: It consists of a hash symbol, followed by an exclamation mark, followed by the path to the interpreter.
- For Bash, this is: `#!/bin/bash`

Bérubé-Vallières © 2026
See last slide for complete credits

Running a script

1. Give yourself execute permissions on the script
2. Type the path of the script to execute it, followed by the flags and arguments
   - e.g. `comp206/myprogram.sh arg1 arg2`

- If the script is in the current directory, you must explicitely specify that it is in the current directory by using `./` at the front
  - e.g. `./myprogram.sh arg1 arg2`

Bérubé-Vallières © 2026
See last slide for complete credits

Example: Writing a script

Want to make a script that will move all my txt files to a directory called textfiles and all my doc files to a directory called docfiles

```
$ vim move.sh
```

(vim)

```
#!/bin/bash
# Move txt and doc files from curr dir
# to new dirs txtfiles and docfiles
mkdir txtfiles
mkdir docfiles
mv *.txt txtfiles
mv *.doc docfiles
```

shebang

comments

```
$ chmod +x move.sh
$ ./move.sh
```


Bérubé-Vallières © 2026
See last slide for complete credits

Exit status

- Any program, when it exits, gives a *status code* back to the OS to signal whether the command succeeded or failed
  - Also called *exit status*
- Code 0 means success, and any other code means failure
  - Commonly, code 1 is used for failure
- You an end a script by using the `exit` command, optionally specifying a status code, e.g. `exit 1`
  - The default code is 0.
- Each time we run a command, either interactively in the terminal, or via a script, the shell updates the variable $? with the exit status of the last command we ran.
  - Can check exit status of previous command with `echo $?`
    - Note: After you do this, the exit status will be replaced by that of the echo command!

Bérubé-Vallières © 2026
See last slide for complete credits

Running commands one after another

- There are multiple ways to chain commands together
  - One way that we saw is redirection/piping
  - Another way that we saw is to write a script, where each line is executed one after the other
- Another way is the semicolon (;), with the same meaning as a new line in a script:
  - `mkdir my_dir ; cd my_dir # runs mkdir my_dir, then cd my_dir`
- A variant of this that is the double ampersand (&&), which executes the second command only if the first one exits successfully
  - `mkdir my_dir && cd my_dir # runs mkdir my_dir, then if mkdir exited with 0, runs cd my_dir`
- Another variant of this that is the double pipe (||), which executes the second command only if the first one does *not* exit successfully

Bérubé-Vallières © 2026
See last slide for complete credits

Bash expressions

Bérubé-Vallières © 2026
See last slide for complete credits

Variables

- Variables are assigned using the syntax `name=value`
  - Important: no space before and after equal sign!
- To use a variable, `$name or ${name}` is used
- Examples:
  - str="hello world"
  - num=5
  - date=$(date)
  - echo $str
  - echo ${num}
  - echo $date

Bérubé-Vallières © 2026
See last slide for complete credits

Positional variables

- Parameters passed to a bash script (e.g. on the command line) are numbered from $1
- For example, if we run `./script 5 2 Bob`
- From the script we can access `5, 2, Bob` using $1, $2, $3

Bérubé-Vallières © 2026
See last slide for complete credits

Other important positional variables

- System parameters:
  - $# : number of arguments on the command line
  - $? : exit value of the last command executed
- Command-line parameters:
  - $n : where n is from 1 through 9, reading left to right
    - ${10} , ${11} …
  - $0 : the name of the current shell or program
  - $@ : all arguments on the command line ("$1", "$2", . . ., "$9") as an array

Bérubé-Vallières © 2026
See last slide for complete credits

Environment variables

- Environment variables contain information about your login session (The time between login and logout)
  - Examples:
    - `$HOME`
    - `$PATH`
    - `$PS1`
- Just like any other variables, they can be edited
  - Often, we want *child processes* (e.g. scripts you run from your Shell) to inherit these variables: the `export` keyword does this
    - `export PATH="$PATH:~/my_dir"`
      - Adds `~/my_dir` to your `$PATH`
  - Variable modifications are only visible inside that particular session

Bérubé-Vallières © 2026
See last slide for complete credits

Bash control structures

Bérubé-Vallières © 2026
See last slide for complete credits

If-statement

```
if COMMAND
then
    ...
elif COMMAND
then
    ...
else
    ...
fi
```


Bérubé-Vallières © 2026
See last slide for complete credits

If statement

```
if COMMAND
then
    ...
elif COMMAND
then
    ...
else
    ...
fi
```

If-statement runs a given command, and if it succeeds (i.e. exits with code 0), it runs the block of code in the `then` branch

Bérubé-Vallières © 2026
See last slide for complete credits

If-statement (equivalently)

```
if COMMAND; then
    ...
elif COMMAND; then
    ...
else
    ...
fi
```

Will often see this written with semicolon instead (remember that the semicolon has the same meaning as a new line in a script!)

Bérubé-Vallières © 2026
See last slide for complete credits

While loop

```
while COMMAND; do
    ...
done
```


Bérubé-Vallières © 2026
See last slide for complete credits

Comparison (`test`)

- Often, we want a condition in an if-statement or while loop to check if two variables are equal, or if a variable is equal to a particular string, etc.
- The command for this is called `test`

Bérubé-Vallières © 2026
See last slide for complete credits

`test` and `[`

```
if test "$1" = "hello" ; then
    echo "Hello World!"
fi


if [ "$1" = "hello" ] ; then
    echo "Hello World!"
fi
```


Bérubé-Vallières © 2026
See last slide for complete credits

`test` and `[`

```
if test "$1" = "hello" ; then
    echo "Hello World!"
fi
```

What arguments does `test` receive?

1. The first string to consider
2. The equals sign, to indicate a string comparison
3. The second string to consider

```
if [ "$1" = "hello" ] ; then
    echo "Hello World!"
fi
```

What arguments does `[` receive?

1. The first string to consider
2. The equals sign, to indicate a string comparison
3. The second string to consider
4. `]`

Spacing is important!

Bérubé-Vallières © 2026
See last slide for complete credits

`test` flags (files)

Use `man test` to learn about the different comparisons you can use

Here are some useful ones to use with files:

- `-f FILE`: if FILE exists and is a regular file
- `-d FILE`: if FILE exists and is a directory
- `-x FILE`: if FILE exists and is executable

Bérubé-Vallières © 2026
See last slide for complete credits

`test` flags (integers)

Use `man test` to learn about the different comparisons you can use

Here are some useful ones to compare integers:

- `N1 -eq N2`: if N1 equals N2
- `N1 -ne N2`: if N1 is not equal to N2
- `N1 -lt N2`: if N1 is less than N2
- `N1 -le N2`: if N1 is less than or equal to N2
- (similar for `gt` and `ge`)

Bérubé-Vallières © 2026
See last slide for complete credits

`test` flags (strings)

Use `man test` to learn about the different comparisons you can use

Here are some useful ones to test strings:

- `-n STRING`: if string length is nonzero
- `-z STRING`: if string length is zero
- `S1 = S2`: if strings S1 and S2 are equal
- `S1 != S2`: if strings S1 and S2 are not equal

Bérubé-Vallières © 2026
See last slide for complete credits

`test` not

Use `man test` to learn about the different comparisons you can use

Can use ! in front of an expression to invert the outcome (condition evaluates to true if expression is false)

Example:

```
if [ ! –f $my_file]; then
        echo “$my_file doesn’t exist!”
fi
```


Bérubé-Vallières © 2026
See last slide for complete credits

For loop

```
for NAME in VALUE1 VALUE2 VALUE3
do
    COMMANDS
done
```

$NAME is set to VALUE1 on first iteration, VALUE2 on second iteration, etc.

Bérubé-Vallières © 2026
See last slide for complete credits

For loop example

```
for f in * ; do
    echo "$f"
done
```

What does this program do?

Bérubé-Vallières © 2026
See last slide for complete credits

Case statement

```
case EXPRESSION in
   PATTERN1)
      ...
   ;;
   PATTERN2)
      ...
   ;;
   PATTERN3)
      …
   ;;
esac
```

if EXPRESSION matches a certain pattern, the corresponding block is executed

Bérubé-Vallières © 2026
See last slide for complete credits

Case statement (example)

```
animal=$1
case $animal in
   *at)
     echo "You entered a word that ends with 'at'"
  ;;
  dog)
     echo "You entered 'dog'"
  ;;
  *)
     echo "You did not enter 'cat' or 'dog'"
  ;;
esac
```


Bérubé-Vallières © 2026
See last slide for complete credits

More on Bash scripts

Bérubé-Vallières © 2026
See last slide for complete credits

`read`

- When we run a script from the terminal, it inherits all the file descriptors of the shell that runs the script.
  - Therefore, the script's stdin is connected to the terminal.
- Bash provides a built-in command called `read` that reads from stdin into a variable.
  - Reminder that you can find info about the built-in commands by typing `man builtins`
- Example:

```
echo "How old are you?"
read -r age
echo "You are $age years old"
```


Bérubé-Vallières © 2026
See last slide for complete credits

`read` as a condition

- The `read` command gives exit status zero when when it successfully reads a line and it didn't reach an *end-of-file* symbol
  - Therefore we can use read as the COMMAND in a while loop to loop over lines of stdin.
- Example:

```
while read -r line ; do
    echo "Here's a line: $line"
done
```


Bérubé-Vallières © 2026
See last slide for complete credits

shift

- This script prints each command-line argument on its own line.

```
while [ $# -gt 0 ] ; do
    echo "$1"
    shift
done
```

- shift is a builtin command that "shifts" over all the numbered variables. That is, the value of $2 moves into $1, the value of $3 moves into $2, and so on. The value of $1 is lost.
- By using shift, we reduce the count of command-line argument by one, which is why $# eventually reaches zero, ending the loop.

Bérubé-Vallières © 2026
See last slide for complete credits

Short-circuiting using && and ||

- We previously saw that && executes the second command only if the first one exits successfully
  - e.g. mkdir my_dir && cd my_dir
    - # will only cd into my_dir if we created my_dir
- Can use this to chain conditions (“AND”)
- Example:

```
if [ -f file1 ] && [ -f file2]; then
   echo "file1 and file2 exist“
else
   echo "missing file1 or file2!"
```


Bérubé-Vallières © 2026
See last slide for complete credits

Short-circuiting using && and ||

- A similar operator is ||, which executes the second command only if the first one did *not* exit succesfully
- Can use this to chain conditions (“OR”)
- Example:

```
if [ $1 = "cat" ] || [ $1 = "dog" ]; then
   echo "you selected cat or dog!"
else
   echo "you did not select cat or dog!"
```


Bérubé-Vallières © 2026
See last slide for complete credits

Functions

- Like "real" programming languages, Bash has functions
  - However, implementation is limited
- Useful whenever there is repetitive code
- Syntax:

```
function_name() {
    commands…
}
```

Other syntax:

```
function function_name() {
    commands…
}
```

- Functions are called by invoking their names, equivalent to calling a command

Bérubé-Vallières © 2026
See last slide for complete credits

Functions (example)

```
#!/bin/bash
hello_world() {
    echo "Hello World!"
}

hello_world # calls hello_world()
```


Bérubé-Vallières © 2026
See last slide for complete credits

Functions: Passing arguments

- Like a command, arguments can be passed to functions, and they can be accessed through `$1, $2, …`
- Example:

```
add() {
    echo $(( $1 + $2 ))
}

add 5 6 # will echo 11
```


Bérubé-Vallières © 2026
See last slide for complete credits

Functions: Local variable

- A variable declared as local is only visible within the block of code in which it appears (local scope)
- Example:

```
fun() {
      local x=5
      y=6
}

fun
echo "Value of x outside function: $x"
# Will print "Value of x outside function:  " ($x is empty)
echo "Value of y outside function: $y"
# Will print "Value of y outside function: 6"
```


Bérubé-Vallières © 2026
See last slide for complete credits

Functions: returning value

- Just like commands, functions return an exit status
  - 0 for success, non-zero for failure
  - Cannot trivially return a value as in “real” programming languages
- Can use command substitution and echo to return a value

```
add() {
      local sum=$(( $1 + $2 ))
      echo $sum
}

result=$(add 5 6) # result is now 11
```

- There are other methods, e.g using global variables, but can be error-prone

Bérubé-Vallières © 2026
See last slide for complete credits

System scripts

- Used by the shell for configuration purposes
- Examples:
  - `~/.bash_profile`: Executed when you login a Bash shell
  - `~/.bashrc`: Executed whenever an interactive non-login Bash shell is opened
- Can use this to set environment variables every time you login!
- Example:
  - Add `export PS1="[\t]\u@\h:\w\$ ` to `~/.bash_profile`

Bérubé-Vallières © 2026
See last slide for complete credits

Alias

- The alias command
  - You give an alternate name to commands
  - Syntax:
    - `alias NEWNAME=OLDEXPRESSION`
  - Example:
    - `alias dir='ls -l -a'`
    - What if you did `alias ls='ls -l -a'` instead?

Bérubé-Vallières © 2026
See last slide for complete credits

Additional commands seen through assignments

- ssh
- scp
- git clone
- git add
  - git add .
- git commit
- git push

Bérubé-Vallières © 2026
See last slide for complete credits

Reminders

Bérubé-Vallières © 2026
See last slide for complete credits

Last reminder

- Class Test 1: Tuesday Oct 6, class time (14:35-15:55)
  - Exam is 75 minutes (1h15)
  - Letter-sized (8.5” by 11”) handwritten crib sheet, single or double-sided
- MyCourses:
  - Practice Midterm 1 from Winter 2026
  - Practice problems written by TA
  - Feel free to discuss both of these on Ed!
- Homework:
  - Reminder that the hard deadline for homework 1 to 4 is Oct 6, the day of Class Test 1!
- Next quiz: Thursday Oct 8

Bérubé-Vallières © 2026
See last slide for complete credits

Remaining tutorials before Class Test 1

- Tutorials:
  - Tutorial 4: Solving a TA Bash practice problem
    - Friday Oct 2 ENGTR 2110, 13:35-14:25 (TA Guru)
    - Monday Oct 5 ENGTR 2120, 10:35-11:25 (TA Michael)
    - Monday Oct 5 ENGTR 1090, 14:35-15:25 (TA Abidur)
    - (Next tutorial after this: Friday Oct 16, Intro to C)

Bérubé-Vallières © 2026
See last slide for complete credits

Remaining study sessions before Class Test 1

- Study sessions (Trottier 3rd floor):
  - Friday Oct 2nd 14:15-15:15
  - Tuesday Oct 6th 13:00-14:00
- Considering working on your assignments and studying for the classes there! Two TAs will be walking around to help you!
  - Check in ENGTR 3104 if you cannot find the TAs!

Bérubé-Vallières © 2026
See last slide for complete credits

Remaining OH before Class Test 1

- My appointment OH: Some times remaining on Monday Oct 5
- TA Mentorship/OH appointments: Many times still available every day until the exam.

Bérubé-Vallières © 2026
See last slide for complete credits